

HỌC VIỆN CÔNG NGHỆ BƯU CHÍNH VIỄN THÔNG
KHOA CÔNG NGHỆ THÔNG TIN



BÁO CÁO BÀI TẬP LỚN
MÔN: CÁC HỆ THỐNG PHÂN TÁN

**Đề tài: Xây dựng hệ thống đặt đồ ăn trực tuyến sử dụng
kiến trúc Microservices.**

Giảng viên:	Kim Ngọc Bách
Nhóm:	03
Danh sách thành viên:	
Lương Trí Tuệ	B22DCCN762
Bùi Ngọc Thiện	B22DCCN822
Trần Bá Thành	B22DCCN798

MỤC LỤC

THUẬT NGỮ VIẾT TẮT.....	4
CHƯƠNG 1. TỔNG QUAN VÀ MÔ TẢ BÀI TOÁN	6
1.1 Bối cảnh và lý do chọn đề tài.....	6
1.2 Mô tả bài toán	6
2.1 Sơ đồ kiến trúc tổng thể	8
2.2 Chiến lược quản trị dữ liệu	10
2.3 Triển khai và Vận hành	10
3.1 Các microservice hoạt động.....	12
3.2 Xử lý khi service tạm thời không phản hồi.....	14
4.1 Nguyên tắc thiết kế api chung.....	16
4.2 Kiến trúc phân hệ API.....	16
CHƯƠNG 5. LUỒNG MESSAGE QUEUE (APACHE KAFKA).....	18
5.1 Cơ sở lý thuyết về Kafka và Consumer Group.....	18
5.2 Kiến trúc xử lý nghiệp vụ phân tán.....	19
5.3 Chi tiết phân luồng và cấu trúc Payload	19
5.4 Cơ chế chịu lỗi	21
5.5 Đánh giá việc áp dụng Apache Kafka.....	21
6.1 Môi trường triển khai và thử nghiệm.....	23
6.2 Kịch bản kiểm thử.....	24
6.3 Đánh giá ưu điểm của hệ thống	2

6.4 Hạn chế và hướng phát triển trong tương lai	3
TÀI LIỆU THAM KHẢO	5
PHỤ LỤC	5

THUẬT NGỮ VIẾT TẮT

Thuật ngữ viết tắt	Nghĩa tiếng Anh	Nghĩa tiếng Việt
API	Application Programming Interface	Giao diện lập trình ứng dụng
CLI	Command Line Interface	Giao diện dòng lệnh (Sử dụng để chạy các lệnh Docker)
CRUD	Create, Read, Update, Delete	Các thao tác cơ bản trong cơ sở dữ liệu: Tạo, Đọc, Cập nhật, Xóa
DDoS	Distributed Denial of Service	Tấn công từ chối dịch vụ phân tán
HTTP	Hypertext Transfer Protocol	Giao thức truyền tải siêu văn bản
JSON	JavaScript Object Notation	Ký pháp đối tượng JavaScript (Định dạng trao đổi dữ liệu)
JWT	JSON Web Token	Chuỗi mã thông báo xác thực an toàn định dạng JSON
REST	Representational State Transfer	Trạng thái chuyển giao đại diện (Kiến trúc phần mềm)
UI/UX	User Interface / User Experience	Giao diện người dùng / Trải nghiệm người dùng
URL	Uniform Resource Locator	Trình định vị tài nguyên thống nhất / Đường dẫn web

DANH MỤC HÌNH ẢNH

Hình 1. Sơ đồ kiến trúc hệ thống.....	8
Hình 2. Giao diện đăng nhập.....	24
Hình 3 Giao diện đặt hàng.....	25
Hình 4 Giao diện quản lý nhà hàng.....	26
Hình 5 Giao diện cập nhật trạng thái đơn hàng.....	28
Hình 6 Giao diện thông báo.....	28
Hình 7 Giao diện Admin Dashboard.....	29
Hình 8 Giao diện đơn hàng của khách.....	30
Hình 9 CSDL sau khi đặt món.....	31
Hình 10 Đơn hàng bị hủy.....	31
Hình 11 CSDL sau khi hủy đơn hàng.....	1

CHƯƠNG 1. TỔNG QUAN VÀ MÔ TẢ BÀI TOÁN

1.1 Bối cảnh và lý do chọn đề tài

Trong kỷ nguyên số hóa hiện nay, dịch vụ đặt đồ ăn trực tuyến đã trở thành một phần thiết yếu trong đời sống hàng ngày, kéo theo lượng truy cập và giao dịch khổng lồ trên các nền tảng thương mại điện tử. Các hệ thống phần mềm truyền thống được xây dựng theo kiến trúc nguyên khối (Monolithic) thường gặp khó khăn trong việc mở rộng quy mô, bảo trì và cập nhật tính năng mới do tính phụ thuộc chặt chẽ giữa các module. Khi một thành phần tải nặng hoặc gặp sự cố, toàn bộ hệ thống có nguy cơ bị gián đoạn.

Để giải quyết triệt để những hạn chế trên, kiến trúc Microservices ra đời như một giải pháp thiết yếu. Việc phân tách một ứng dụng lớn thành các dịch vụ nhỏ (microservices) hoạt động độc lập giúp hệ thống dễ dàng mở rộng (scale) theo nhu cầu thực tế của từng nghiệp vụ, tăng cường khả năng chịu lỗi (fault tolerance) và đẩy nhanh tốc độ phát triển phần mềm. Việc áp dụng mô hình Microservices kết hợp với các công nghệ giao tiếp hiện đại như RESTful API và Message Queue là yêu cầu cấp thiết để xây dựng một nền tảng đặt đồ ăn trực tuyến ổn định, linh hoạt và hiệu suất cao.

1.2 Mô tả bài toán

Mục tiêu của đề tài là xây dựng một **Hệ thống đặt đồ ăn trực tuyến** tuân thủ nghiêm ngặt các nguyên tắc của kiến trúc Microservices. Hệ thống đóng vai trò cầu nối giữa thực khách và các nhà hàng, cho phép người dùng tìm kiếm, duyệt thực đơn và đặt món, đồng thời hỗ trợ chủ nhà hàng quản lý thông tin kinh doanh và lượng hàng tồn kho.

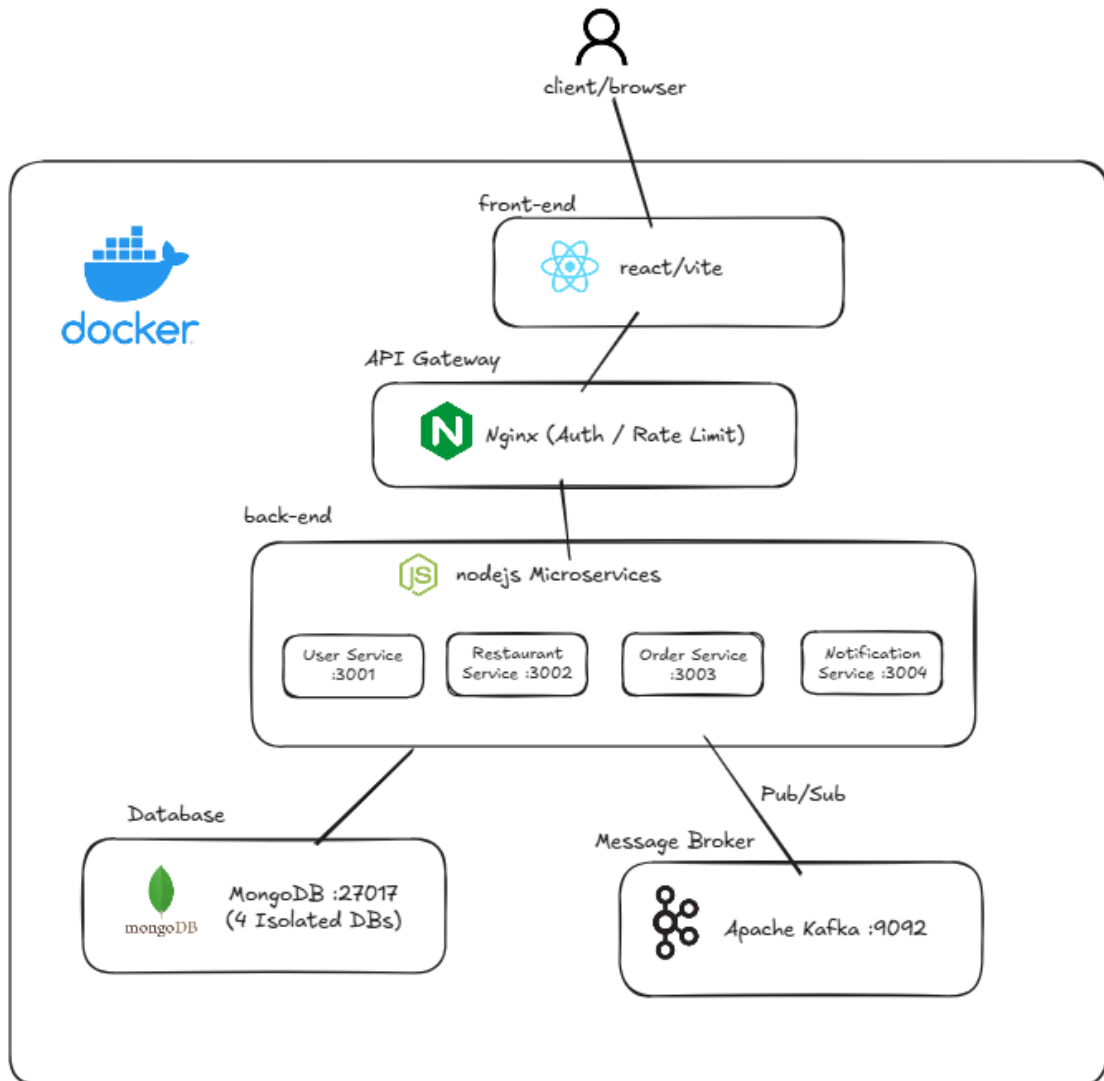
Để đáp ứng yêu cầu kỹ thuật phân tán, bài toán đặt ra các ràng buộc thiết kế cốt lõi sau:

- **Phân tách miền nghiệp vụ:** Ứng dụng phải được chia nhỏ thành các dịch vụ chạy độc lập, mỗi dịch vụ phụ trách một miền chức năng riêng. Hệ thống được thiết kế với 5 khối chính: API Gateway, User Service, Restaurant Service, Order Service và Notification Service.

- **Cơ sở dữ liệu độc lập:** Mỗi microservice phải sở hữu một cơ sở dữ liệu riêng biệt (sử dụng MongoDB) để đảm bảo tính đóng gói, độc lập dữ liệu và nguyên tắc không chia sẻ trạng thái.
- **Giao tiếp đồng bộ:** Các dịch vụ cung cấp các endpoint RESTful API để trao đổi dữ liệu trực tiếp với nhau và xử lý request từ client.
- **Giao tiếp bất đồng bộ qua Message Queue:** Hệ thống yêu cầu tích hợp Message Broker (cụ thể là Kafka) để xử lý các luồng sự kiện phân tán. Bài toán yêu cầu xử lý luồng tồn kho tự động: khi order-service phát hành (publish) sự kiện tạo đơn hàng, restaurant-service sẽ tiêu thụ (consume) sự kiện này để tự động trừ số lượng món ăn tồn kho mà không khóa luồng xử lý chính.
- **Triển khai đồng bộ:** Toàn bộ kiến trúc đa dịch vụ phải được container hóa và khởi chạy tự động thông qua công cụ Docker Compose.

CHƯƠNG 2: KIẾN TRÚC HỆ THỐNG

2.1 Sơ đồ kiến trúc tổng thể



Hình 1. Sơ đồ kiến trúc hệ thống

Hệ thống được thiết kế theo mô hình kiến trúc vi dịch vụ (Microservices Architecture), phân tách các chức năng thành các thành phần độc lập nhằm tối ưu hóa khả năng mở rộng, bảo trì và tăng tính chịu lỗi. Về mặt tổng thể, kiến trúc hệ thống được phân thành ba tầng (layers) biểu diễn rõ rệt:

- **Tầng API Gateway (Tầng Biên):** Hoạt động trên cổng 3000, đây là điểm tiếp nhận duy nhất (Single Point of Entry) cho mọi yêu cầu (request) xuất phát từ phía Client. API Gateway đóng vai trò như một Reverse Proxy, làm

nhiệm vụ định tuyến (routing) linh hoạt các yêu cầu HTTP đến đúng dịch vụ nội bộ tương ứng. Ngoài ra, tầng này còn chịu trách nhiệm xử lý các tác vụ cắt ngang (cross-cutting concerns) cốt lõi như xác thực người dùng tập trung bằng JSON Web Token (JWT Auth) và kiểm soát lưu lượng mạng để ngăn chặn quá tải thông qua cơ chế giới hạn truy cập (rate limit).

- **Tầng Dịch vụ (Microservices Layer):** Là nơi chứa đựng logic nghiệp vụ của hệ thống, bao gồm 4 vi dịch vụ hoàn toàn độc lập chạy trên các công mạng khác nhau. Mỗi dịch vụ tự đóng gói một miền nghiệp vụ riêng biệt và tương tác với nhau thông qua API hoặc sự kiện:
 - **User Service (Cổng 3001):** Quản lý định danh, đăng ký, đăng nhập và hồ sơ người dùng.
 - **Restaurant Service (Cổng 3002):** Quản lý thông tin nhà hàng, thực đơn và số lượng tồn kho.
 - **Order Service (Cổng 3003):** Chịu trách nhiệm khởi tạo đơn hàng và xuất bản các sự kiện liên quan đến vòng đời đơn hàng.
 - **Notification Service (Cổng 3004):** Hệ thống tiêu thụ sự kiện để thiết lập thông báo cho người dùng.
- **Tầng Hạ tầng và Broker (Infrastructure & Message Broker):** Cung cấp nền tảng lưu trữ và giao tiếp bất đồng bộ cho các dịch vụ.
 - Hệ quản trị cơ sở dữ liệu phi quan hệ **MongoDB (Cổng 27017)** được sử dụng để lưu trữ dữ liệu vĩnh viễn cho toàn bộ các dịch vụ.
 - Nền tảng luồng sự kiện phân tán **Apache Kafka (Cổng 9092)** đóng vai trò là Message Broker trung tâm. Nó chịu trách nhiệm phân phối các thông điệp sự kiện (như order-events) giữa các dịch vụ để đảm bảo quá trình xử lý bất đồng bộ diễn ra nhanh chóng, giảm thiểu độ trễ cho người dùng.

2.2 Chiến lược quản trị dữ liệu

Để đảm bảo tính độc lập tối đa và tránh tình trạng thắt nút cổ chai dữ liệu (database bottleneck), kiến trúc hệ thống áp dụng triệt để mô hình thiết kế "Mỗi dịch vụ một cơ sở dữ liệu" (Database-per-service).

Mặc dù MongoDB được sử dụng làm công cụ quản trị cơ sở dữ liệu chung, tuy nhiên dữ liệu không được đặt chung trong một schema lớn. Thay vào đó, dữ liệu được phân chia rạch ròi thành các cơ sở dữ liệu logic (logic databases) riêng biệt cho từng miền nghiệp vụ. Cụ thể:

- **users_db:** Được toàn quyền kiểm soát bởi User Service, lưu trữ thông tin nhạy cảm như tài khoản, mật khẩu, và vai trò (role) của người dùng.
- **restaurants_db:** Thuộc quyền sở hữu của Restaurant Service, dùng để lưu trữ danh mục nhà hàng, chi tiết các món ăn và trạng thái tồn kho hiện tại.
- **orders_db:** Nơi Order Service lưu vết lịch sử giao dịch, chi tiết các món đã đặt và trạng thái theo dõi của từng đơn hàng.
- **notifications_db:** Được Notification Service sử dụng để ghi nhận nội dung và trạng thái đọc/chưa đọc của các thông báo hệ thống.

Chiến lược phân tách này triệt tiêu hoàn toàn các thao tác truy vấn chéo (cross-query) trực tiếp vào cơ sở dữ liệu của dịch vụ khác. Mọi nhu cầu trao đổi dữ liệu bắt buộc phải thông qua RESTful API hoặc Kafka events, từ đó nâng cao tính bảo mật và khả năng mở rộng độc lập của từng module.

2.3 Triển khai và Vận hành

Quá trình triển khai và vận hành hệ thống được chuẩn hóa và tự động hóa cao độ thông qua nền tảng ảo hóa container Docker. Cách tiếp cận này đảm bảo tính nhất quán của môi trường chạy mã (runtime environment) từ máy tính của lập trình viên lên đến máy chủ thực tế.

- **Điều phối bằng Docker Compose:** Toàn bộ cấu trúc liên kết của hệ thống được định nghĩa trong một tập tin cấu hình duy nhất là docker-

compose.yml. Tập tin này thiết lập một mạng nội bộ chung, cho phép các container của từng dịch vụ dễ dàng tìm thấy và giao tiếp với nhau qua tên dịch vụ thay vì địa chỉ IP cứng. Đồng thời, nó quy định rõ ràng thứ tự khởi động (ví dụ: các microservices phải đợi MongoDB và Kafka khởi chạy thành công) và khai báo các Volumes để đảm bảo dữ liệu trong MongoDB không bị mất đi khi container bị xóa (persistent storage).

- **Quy trình khởi chạy:** Nhờ vào kiến trúc đóng gói này, toàn bộ khối hệ thống phức tạp bao gồm API Gateway, 4 microservices lõi, cơ sở dữ liệu MongoDB và bộ môi giới thông điệp Kafka đều có thể được tự động đóng gói (build) và chạy ngầm (detached mode) đồng bộ chỉ bằng một câu lệnh Command Line Interface (CLI) duy nhất: `docker compose up -d --build`. Khi cần dừng toàn bộ hệ thống, lệnh `docker compose down` sẽ dọn dẹp tài nguyên một cách an toàn.

CHƯƠNG 3. MÔ TẢ CÁC MICROSERVICE

3.1 Các microservice hoạt động

Hệ thống đặt đồ ăn trực tuyến được thiết kế và phát triển dựa trên kiến trúc Microservices. Để tăng tính độc lập, khả năng mở rộng và khả năng bảo trì, toàn bộ hệ thống được phân tách thành 5 dịch vụ thành phần (bao gồm 1 API Gateway và 4 dịch vụ Back-end lõi). Mỗi dịch vụ phụ trách một miền nghiệp vụ độc lập và tuân thủ nguyên lý đơn trách nhiệm.

Giao tiếp nội bộ giữa các dịch vụ được thực hiện thông qua giao thức HTTP RESTful API đối với các tiến trình đồng bộ và thông qua Apache Kafka (Message Queue) đối với các tiến trình bất đồng bộ theo mô hình hướng sự kiện.

Dưới đây là mô tả chức năng và vai trò của từng microservice trong hệ thống:

1. API Gateway - Cổng 3000

Vai trò: API Gateway hoạt động như điểm truy cập duy nhất cho toàn bộ yêu cầu từ phía máy khách và giao diện front-end.

Chức năng: Thực hiện định tuyến các yêu cầu HTTP đến các microservice nghiệp vụ tương ứng ở lớp phía sau. Ngoài chức năng định tuyến, dịch vụ còn đảm nhiệm các tác vụ kiểm soát an ninh mạng cốt lõi bao gồm xác thực quyền truy cập tập trung bằng JSON Web Token (JWT Auth) và giới hạn lưu lượng truy cập (Rate Limit) nhằm bảo vệ hệ thống khỏi tình trạng quá tải.

2. User Service - Cổng 3001

Vai trò: Hoạt động như trung tâm quản lý định danh, phụ trách toàn bộ vòng đời của tài khoản người dùng và hệ thống phân quyền.

Chức năng: Cung cấp các giao diện lập trình (RESTful API) để thực hiện đăng ký tài khoản (dành cho thực khách và chủ nhà hàng), xác thực đăng

nhập (cấp phát token JWT) và quản lý hồ sơ cá nhân (user profile). Dịch vụ này lưu trữ và quản lý dữ liệu độc quyền trên cơ sở dữ liệu `users_db`.

3. Restaurant Service - Cổng 3002

Vai trò: Phụ trách quản lý toàn bộ thông tin về các đối tác nhà hàng, thực đơn kinh doanh và kiểm soát lượng hàng tồn kho (stock).

Chức năng: Cung cấp các API để tạo mới nhà hàng, thêm món ăn và thiết lập thực đơn lưu tại `restaurants_db`. Đặc biệt, dịch vụ hoạt động như một Consumer, lắng nghe thông điệp sự kiện từ Kafka (order-events) để tự động trừ số lượng tồn kho khi có đơn đặt hàng mới (ORDER_CREATED) hoặc cộng hoàn lại tồn kho khi đơn bị hủy (ORDER_CANCELLED), đồng thời cập nhật trạng thái hết hàng nếu số lượng giảm về 0.

4. Order Service - Cổng 3003

Vai trò: Đóng vai trò là lõi nghiệp vụ của hệ thống, điều phối và quản lý toàn bộ vòng đời xử lý của một đơn đặt đồ ăn.

Chức năng: Tiếp nhận yêu cầu đặt hàng, ghi nhận chi tiết món ăn, tính tổng tiền và cập nhật trạng thái đơn hàng vào `orders_db`. Đồng thời, dịch vụ hoạt động như một Publisher, tự động xuất bản (publish) các sự kiện thay đổi (như tạo mới hoặc hủy đơn) lên hệ thống Message Broker (Apache Kafka) để kích hoạt các tiến trình xử lý bất đồng bộ ở các dịch vụ liên quan.

5. Notification Service - Cổng 3004

Vai trò: Hoạt động như một hệ thống thông báo tập trung, cung cấp các cập nhật kịp thời về trạng thái xử lý đơn hàng cho người dùng cuối.

Chức năng: Đóng vai trò là một Consumer độc lập, liên tục tiêu thụ các sự kiện liên quan đến đơn hàng từ Kafka. Dựa trên các thông điệp này, dịch vụ tự động biên dịch thành các bản tin thông báo (notification), lưu trữ vào `notifications_db` và cung cấp API để front-end có thể truy xuất, hiển thị trạng thái đã đọc/chưa đọc cho người dùng.

3.2 Xử lý khi service tạm thời không phản hồi

Trong kiến trúc phân tán nói chung và kiến trúc microservices nói riêng, việc một dịch vụ tạm thời không phản hồi do khởi động lại, quá tải hệ thống hoặc sự cố mạng nội bộ là tình huống thường gặp. Để tăng tính bền bỉ của hệ thống và hạn chế hiện tượng lỗi dây chuyền, hệ thống đặt đồ ăn trực tuyến được thiết kế với các cơ chế phòng vệ như sau:

1. Cơ chế kiểm soát thời gian chờ Timeout và Health Check Tất cả yêu cầu nội bộ giữa các dịch vụ thông qua giao thức HTTP đều được kiểm soát chặt chẽ. Tại API Gateway, HTTP Client được cấu hình để định tuyến các request, đồng thời hệ thống triển khai endpoint /health trên từng microservice để liên tục theo dõi tình trạng hoạt động (uptime). Nhờ đó, nếu một dịch vụ đích (downstream service) bị quá tải hoặc mất kết nối, Gateway có thể nhanh chóng nhận biết, giải phóng kết nối và trả về phản hồi lỗi phù hợp cho Front-end (ví dụ: HTTP 503 Service Unavailable) thay vì treo tiến trình vô thời hạn.

2. Cơ chế thử lại (Retry) cho các lỗi tạm thời Khi hệ thống gặp các ngoại lệ về mạng (như lỗi kết nối tạm thời), cơ chế xử lý lỗi cục bộ sẽ tiến hành thử lại (retry) một số lần nhất định. Nếu số lần thử vượt quá ngưỡng cho phép mà dịch vụ đích vẫn không phản hồi, tiến trình sẽ được chuyển hướng sang các kịch bản bắt lỗi (Error Handling) để trả thông báo minh bạch cho người dùng, tránh tình trạng Client phải chờ đợi không có phản hồi.

3. Cơ chế bền bỉ thông điệp (Message Persistence) qua Kafka Đối với các luồng nghiệp vụ liên dịch vụ phức tạp (như xử lý đơn hàng và trừ tồn kho), hệ thống loại bỏ hoàn toàn rủi ro gián đoạn bằng cách thay thế giao tiếp HTTP đồng bộ bằng giao tiếp bất đồng bộ qua Apache Kafka.

- **Trường hợp hoạt động bình thường:** Dịch vụ tiêu thụ (ví dụ: Restaurant Service) nhận và xử lý sự kiện ORDER_CREATED ngay lập tức.
- **Trường hợp dịch vụ tiêu thụ tạm thời không khả dụng:** Nếu Restaurant Service hoặc Notification Service gặp sự cố ngưng hoạt động, các thông điệp sự kiện không hề bị mất đi mà vẫn được lưu trữ an toàn và bền vững bên trong topic order-events của cụm Kafka. Ngay khi dịch

vụ khởi động lại thành công, nó sẽ tự động tiêu thụ (consume) tiếp các thông điệp còn tồn đọng từ vị trí cuối cùng (offset) để thực hiện bù trừ số lượng tồn kho (stock).

Sự kết hợp giữa cơ chế Timeout ở giao tiếp đồng bộ và tính năng lưu trữ thông điệp bền bỉ của Kafka ở giao tiếp bất đồng bộ giúp hệ thống duy trì tính ổn định, đảm bảo tính nhất quán dữ liệu (Eventual Consistency) ngay cả khi xuất hiện sự cố cục bộ.

CHƯƠNG 4. THIẾT KẾ RESTFUL API

4.1 Nguyên tắc thiết kế api chung

Hệ thống tuân thủ nghiêm ngặt chuẩn kiến trúc RESTful.

Toàn bộ API được định tuyến tập trung qua API Gateway, sử dụng phiên bản thống nhất (tiền tố /api/).

Định dạng dữ liệu trao đổi mặc định: JSON. Các mã trạng thái HTTP (200, 201, 400, 401, 403, 404, 500) được chuẩn hóa cho toàn bộ các dịch vụ.

4.2 Kiến trúc phân hệ API

Hệ thống được tổ chức theo kiến trúc microservices với sự phân tách rõ ràng giữa các miền nghiệp vụ. Để bảo đảm tính thống nhất trong quá trình giao tiếp, toàn bộ yêu cầu từ phía máy khách đều được định tuyến tập trung thông qua API Gateway. Cách tiếp cận này giúp tách biệt cấu trúc mạng nội bộ khỏi phía Client, đồng thời chuẩn hóa cơ chế xử lý lỗi và bảo mật bằng JWT.

Bảng tổng quan dưới đây mô tả danh mục các dịch vụ cùng những nhóm nghiệp vụ cốt lõi do phân hệ API của hệ thống đảm nhiệm

Bảng 4.1 Danh mục các vi dịch vụ và nhóm nghiệp vụ cốt lõi

STT	Dịch vụ	Cụm nghiệp vụ đảm nhận	Đối tượng tương tác / Vai trò chính
1	API Gateway (Port 3000)	- Điểm vào tập trung của toàn bộ hệ thống back-end.- Thực hiện định tuyến request đến 4 vi dịch vụ phía sau.- Đảm nhiệm xác thực JWT và giới hạn tốc độ (Rate Limit).	Máy khách Frontend, hệ thống vi dịch vụ nội bộ

2	User Service (Port 3001)	- Quản lý đăng ký tài khoản (khách hàng, chủ nhà hàng) và đăng nhập. - Quản lý thông tin hồ sơ tài khoản người dùng (Profile). - Cung cấp API nội bộ để các dịch vụ khác trích xuất thông tin user.	Người dùng hệ thống, API Gateway
3	Restaurant Service (Port 3002)	- Tạo mới và quản lý hồ sơ nhà hàng cùng trạng thái hoạt động. - Cung cấp API quản lý thực đơn (thêm món, xóa món, định giá). - Quản lý và tự động trừ/hoàn số lượng tồn kho (stock) dựa trên message queue từ Kafka	Chủ nhà hàng, người mua (xem menu), Kafka Broker
4	Order Service (Port 3003)	- Xử lý luồng đặt hàng, tiếp nhận danh sách món và tính tổng tiền. - Quản lý lịch sử và cập nhật trạng thái vòng đời đơn hàng (pending, confirmed, cancelled). - Phát hành sự kiện (ORDER_CREATED, ORDER_CANCELLED) lên Kafka.	Người mua, API Gateway, Kafka Broker
5	Notification Service (Port 3004)	- Lắng nghe sự kiện Kafka để tạo bản ghi thông báo tương ứng với trạng thái đơn hàng. - Cung cấp API truy xuất danh sách thông báo và đánh dấu trạng thái đã đọc cho người dùng.	Người mua, Kafka Broker

CHƯƠNG 5. LƯỒNG MESSAGE QUEUE (APACHE KAFKA)

Trong kiến trúc microservices của nền tảng đặt đồ ăn trực tuyến, cơ chế giao tiếp đồng bộ thông qua HTTP REST API có thể làm gia tăng độ trễ xử lý, tạo điểm nghẽn hiệu năng và ảnh hưởng đến tính sẵn sàng của hệ thống khi có lượng lớn đơn hàng đổ về cùng lúc. Để giảm sự phụ thuộc trực tiếp (tight coupling) giữa các dịch vụ, hệ thống triển khai **Apache Kafka** như một hệ thống môi giới thông điệp (Message Broker) phục vụ giao tiếp bất đồng bộ giữa các thành phần nghiệp vụ.

5.1 Cơ sở lý thuyết về Kafka và Consumer Group

Apache Kafka là một nền tảng truyền phát sự kiện phân tán (distributed event streaming platform), cho phép lưu trữ các thông điệp dưới dạng nhật ký (commit log) bền vững theo trình tự thời gian.

Trong kiến trúc của hệ thống, Kafka hỗ trợ cơ chế **Consumer Group** nhằm tối ưu hóa việc phân phối và xử lý thông điệp:

- Khác với mô hình phát sóng (broadcast) truyền thông, Consumer Group cho phép phân bổ tải xử lý (load balancing) giữa các bản sao (instances) khác nhau của cùng một dịch vụ.
- Nhiều microservice khác nhau (ví dụ: Restaurant Service và Notification Service) có thể cùng đăng ký làm consumer trên một chủ đề (topic) thông qua các Consumer Group độc lập. Điều này đảm bảo mỗi dịch vụ đều nhận được bản sao của thông điệp để thực hiện các nghiệp vụ riêng biệt mà không can thiệp lẫn nhau.
- Kafka duy trì một con trỏ (offset) cho mỗi Consumer Group. Khi một consumer xử lý thành công, nó sẽ gửi lệnh xác nhận (commit offset) lại cho Kafka. Cơ chế này giúp bảo đảm tính toàn vẹn dữ liệu, ngăn ngừa tình trạng mất sự kiện (event loss) nếu dịch vụ gặp sự cố ngắt kết nối giữa chừng.

5.2 Kiến trúc xử lý nghiệp vụ phân tán

Một trong những thách thức chính của bài toán đặt đồ ăn là tính nhất quán dữ liệu chéo giữa các dịch vụ: Order Service cần trừ tồn kho món ăn tại Restaurant Service trong quá trình khởi tạo đơn hàng. Nếu sử dụng API đồng bộ, hệ thống sẽ phát sinh độ trễ cao và người dùng phải chờ đợi lâu.

Để giải quyết vấn đề này, hệ thống triển khai mô hình hướng sự kiện (Event-Driven Architecture) thông qua Kafka:

- **Bên phát hành (Publisher - Source of Truth):** Khi một giao dịch đặt hàng diễn ra thành công tại Order Service, thay vì gọi API trừ kho trực tiếp, dịch vụ này chỉ cần đóng gói thông tin và xuất bản (publish) một sự kiện vào Topic dùng chung. Sau đó, hệ thống lập tức trả phản hồi thành công cho người dùng.
- **Bên tiêu thụ (Consumers):** Các dịch vụ như Restaurant Service và Notification Service hoạt động như những tiến trình ngầm (background workers), liên tục lắng nghe Topic. Khi bắt được sự kiện, chúng sẽ tự động bóc tách payload để cập nhật trạng thái kho và khởi tạo thông báo mà không làm gián đoạn trải nghiệm của người dùng cuối.

5.3 Chi tiết phân luồng và cấu trúc Payload

Để bảo đảm tính module hóa, mọi trạng thái vòng đời của đơn hàng được tập trung truyền tải qua một Topic duy nhất mang tên order-events.

5.3.1 Sự kiện khởi tạo đơn hàng (*ORDER_CREATED*)

Publisher: Order Service

Mục đích: Thông báo hệ thống có một đơn đặt đồ ăn mới cần được xử lý.

Luồng xử lý tại Consumer: * Restaurant Service tiếp nhận sự kiện, trích xuất danh sách các menuItemId và tự động giảm giá trị stock tương ứng. Nếu trường stock giảm về 0, hệ thống tự động cập nhật cờ isAvailable

= false để bảo vệ tính toàn vẹn dữ liệu, ngăn khách hàng khác tiếp tục đặt món.

- Notification Service đồng thời tiếp nhận sự kiện để tạo một bản ghi thông báo gửi đến tài khoản của chủ nhà hàng và người mua.

Ví dụ cấu trúc payload định dạng JSON:

```
{  
  "eventId": "evt_98765",  
  "eventType": "ORDER_CREATED",  
  "timestamp": "2026-05-27T16:28:15Z",  
  "data": {  
    "orderId": "ord_12345",  
    "userId": "user_789",  
    "restaurantId": "rest_001",  
    "items": [  
      { "menuItemId": "item_1", "quantity": 2 }  
    ]  
  }  
}
```

5.3.2 Sự kiện hủy đơn hàng (*ORDER_CANCELLED*)

Publisher: Order Service

Mục đích: Kích hoạt luồng đền bù dữ liệu (Compensating Transaction) khi đơn hàng bị hủy bỏ.

Luồng xử lý tại Consumer: Restaurant Service phân tích payload và tự động hoàn lại số lượng stock đã trừ trước đó, đảm bảo hàng hóa được trả lại trạng thái sẵn sàng bán.

5.3.3 Sự kiện thay đổi trạng thái (*ORDER_STATUS_CHANGED*)

Mục đích: Cập nhật tiến trình của đơn (ví dụ: đang chuẩn bị, đang giao).

Luồng xử lý tại Consumer: Notification Service tiêu thụ sự kiện này để gửi cảnh báo thời gian thực (real-time notification) cho thực khách.

5.4 Cơ chế chịu lỗi

Sự cố mạng nội bộ hoặc quá tải có thể làm gián đoạn quá trình đọc thông điệp của Consumer. Kafka hỗ trợ các cơ chế bảo vệ mạnh mẽ:

1. **Cơ chế thử lại (Retry & Offset Management):** Consumer chỉ xác nhận (commit offset) khi luồng nghiệp vụ ghi vào cơ sở dữ liệu MongoDB hoàn tất thành công. Nếu phát sinh lỗi, hệ thống không commit và sẽ thực hiện thử lại (retry) thông điệp đó trong lần lấy (poll) tiếp theo.
2. **Lưu trữ bền bỉ (Persistence):** Khác với các hệ thống In-memory, Kafka lưu thông điệp trực tiếp xuống ổ cứng vật lý (disk-based storage). Do đó, nếu Restaurant Service bị sập nguồn, các thông điệp *ORDER_CREATED* vẫn nằm an toàn trên Kafka. Khi dịch vụ khởi động lại, nó sẽ tiếp tục xử lý các đơn hàng đang dang dở mà không hề mất mát dữ liệu.

5.5 Đánh giá việc áp dụng Apache Kafka

So với việc gọi REST API truyền thống hay sử dụng các công cụ Pub/Sub cơ bản, Apache Kafka được hệ thống ưu tiên lựa chọn nhờ các đặc điểm kiến trúc vượt trội:

- **Tính phân tách cao (Decoupling):** Các microservice hoàn toàn không cần biết đến sự tồn tại của nhau. Dịch vụ đặt hàng vẫn hoạt động bình thường ngay cả khi dịch vụ thông báo đang được bảo trì.
- **Tính lưu vết và Replay dữ liệu:** Kafka giữ lại các sự kiện trong một khoảng thời gian cấu hình sẵn. Nếu phát hiện logic trừ kho bị lỗi, đội

ngữ phát triển có thể thiết lập lại con trỏ (offset) để chạy lại (replay) toàn bộ luồng sự kiện cũ và khắc phục dữ liệu.

- **Khả năng thông lượng cao (High Throughput):** Được thiết kế để xử lý hàng triệu thông điệp, Kafka loại bỏ hoàn toàn hiện tượng thắt nút cổ chai (bottleneck) khi nền tảng tung ra các đợt khuyến mãi khiến lượng truy cập tăng vọt.

CHƯƠNG 6. THỬ NGHIỆM VÀ ĐÁNH GIÁ

6.1 Môi trường triển khai và thử nghiệm

Để đảm bảo tính đồng nhất và khả năng tái lập môi trường triển khai, toàn bộ hệ thống được đóng gói bằng Docker và quản lý thông qua Docker Compose. Việc sử dụng mô hình container hóa giúp các thành phần của hệ thống hoạt động độc lập nhưng vẫn dễ dàng giao tiếp với nhau trong cùng một môi trường mạng nội bộ.

Cấu hình môi trường thử nghiệm chi tiết:

- Hệ điều hành nền tảng: Windows có tích hợp Docker Engine và Docker Compose.
- Kiến trúc mạng nội bộ (Docker Network): Các container được cô lập và giao tiếp với nhau qua mạng ảo mang tên app-network (bridge driver). Nhằm đảm bảo bảo mật, API Gateway Nginx (chạy tại cổng 3000) là điểm giao tiếp duy nhất tiếp nhận yêu cầu API từ bên ngoài. Ứng dụng Frontend (chạy tại cổng 5173) phục vụ giao diện người dùng và tự động chuyển tiếp các lệnh gọi /api/... tới API Gateway qua mạng nội bộ Docker. Không có Microservice nghiệp vụ nào được truy cập trực tiếp mà không qua Gateway.
- Hệ sinh thái dịch vụ hạ tầng:
 - MongoDB 7: Cơ sở dữ liệu NoSQL chính, mỗi microservice sở hữu một database logic riêng biệt (food_users_db, food_restaurants_db, food_orders_db, food_notifications_db). Cơ sở dữ liệu được lưu trữ lâu bền trên Docker Volume (mongo_data).
 - Apache Kafka (Confluent 7.5.0): Đóng vai trò Message Broker trung tâm cho kiến trúc Event-Driven. Kafka kết hợp với Zookeeper để quản lý metadata cluster. Giao tiếp bất đồng bộ giữa các microservice được thực hiện qua topic order-events.

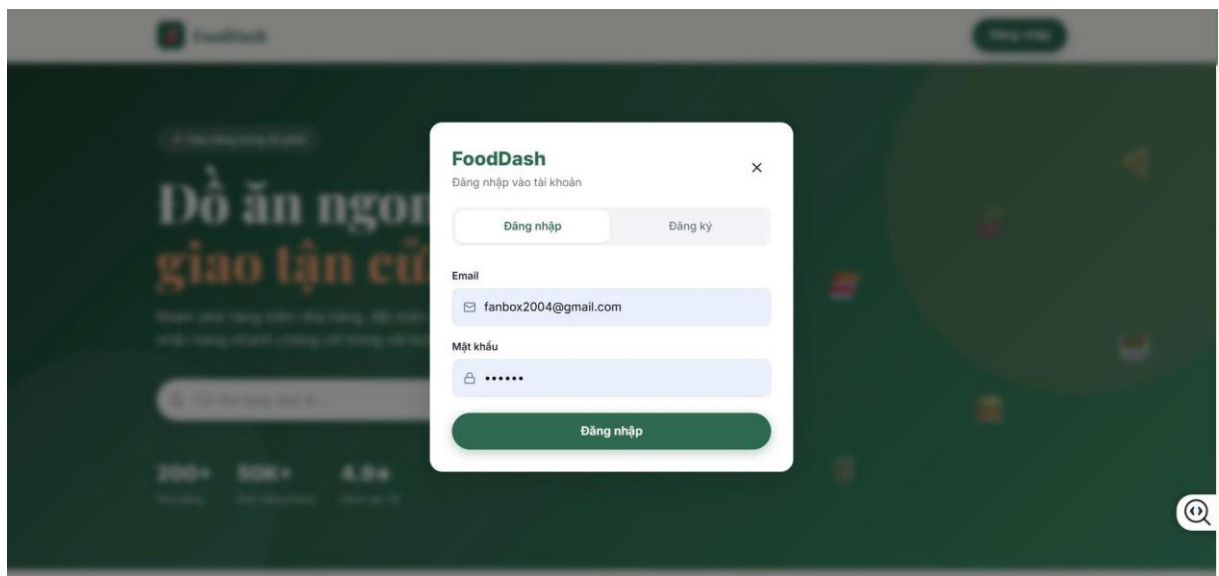
Quá trình khởi động toàn bộ cụm hệ thống (bao gồm Frontend, API Gateway Nginx, 4 Microservices nghiệp vụ, MongoDB, Zookeeper, Kafka và Mongo Express — tổng cộng **10 container**) được tự động hóa hoàn toàn thông qua tập lệnh: `docker-compose up -d --build`.

6.2 Kịch bản kiểm thử

Quá trình kiểm thử được chia thành hai hạng mục chính: kiểm thử tính đúng đắn của luồng nghiệp vụ và kiểm thử tính bền bỉ của kiến trúc phân tán.

6.2.1 Kiểm thử chức năng nghiệp vụ

Luồng xác thực và quản lý định danh:

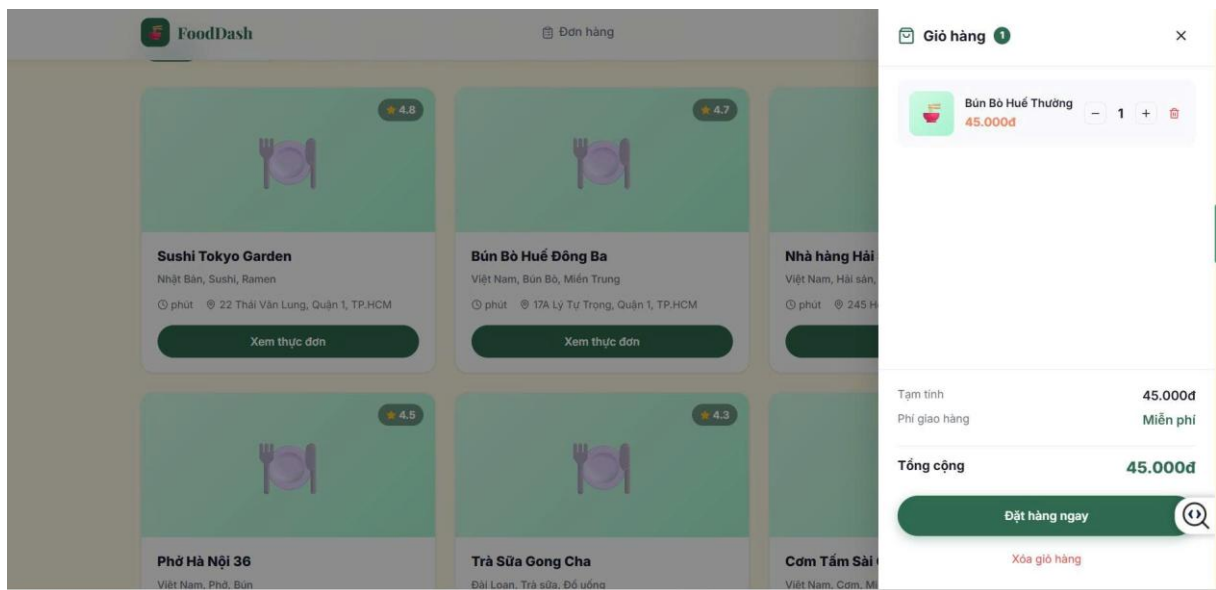


Hình 2. Giao diện đăng nhập

- **Kịch bản:** Người dùng mới truy cập trang chủ tại `http://localhost`, nhấn nút "Đăng nhập" trên thanh Header, hệ thống hiển thị cửa sổ modal xác thực với hai tab "Đăng nhập" và "Đăng ký". Người dùng chuyển sang tab "Đăng ký", nhập đầy đủ thông tin (Họ tên, Email, Mật khẩu tối thiểu 6 ký tự) và nhấn nút "Đăng ký". Sau khi đăng ký thành công, hệ thống tự động đăng nhập và cấp JWT Token.
- **Đánh giá phân quyền:** Sau khi đăng nhập bằng tài khoản khách hàng (customer), cố tình truy cập trực tiếp đường dẫn `/dashboard` (trang quản trị dành riêng cho Chủ nhà hàng và Admin).

- **Kết quả mong đợi:** Hệ thống chặn truy cập, tự động chuyển hướng người dùng về trang chủ do kiểm tra vai trò restaurant_owner hoặc admin không khớp.
- **Kết quả thực tế:** Khách hàng không thể truy cập giao diện quản trị. Header thanh điều hướng chỉ hiển thị các mục phù hợp với vai trò: nút "Đơn hàng" và biểu tượng giỏ hàng cho khách hàng; nút "Dashboard" cho chủ nhà hàng và admin.

Luồng duyệt nhà hàng, đặt hàng và thanh toán



Hình 3 Giao diện đặt hàng

- **Kịch bản:** Khách hàng đã đăng nhập truy cập trang chủ. Trang chủ tự động tải và hiển thị các phần: Banner Hero với ô tìm kiếm, mục "Các bước đặt hàng", danh sách "Món ăn bán chạy" (Best Sellers), lưới các nhà hàng có thể lọc theo loại ẩm thực, và mục "Đối tác nổi bật" (Top Brands).

Bước 1 — Duyệt nhà hàng và xem thực đơn: Khách hàng cuộn xuống phần danh sách nhà hàng, nhấn nút "Xem Menu" trên thẻ nhà hàng mong muốn. Hệ thống hiển thị cửa sổ modal thực đơn với đầy đủ thông tin từng món ăn: tên, mô tả, giá bán, trạng thái tồn kho.

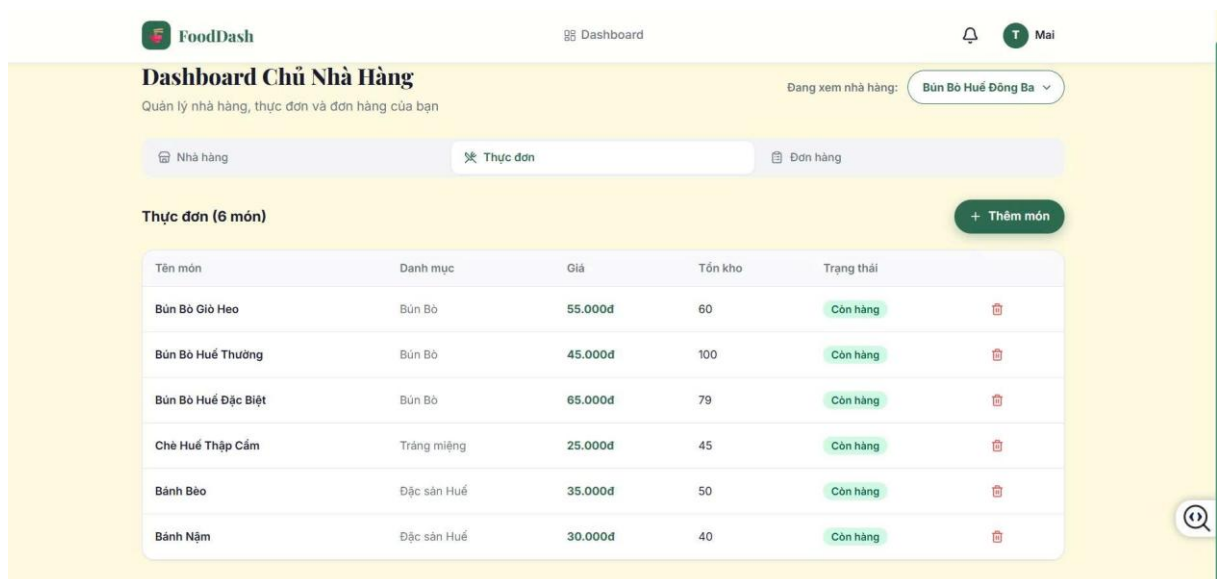
Bước 2 — Thêm món vào giỏ hàng: Khách hàng nhấn nút "Thêm vào giỏ" trên các món ăn mong muốn. Biểu tượng giỏ hàng trên Header tự động cập nhật số lượng. Nhấn vào biểu tượng giỏ hàng để mở ngăn kéo bên phải

(Cart Drawer), hiển thị danh sách các món đã chọn, cho phép tăng/giảm số lượng hoặc xóa từng món. Phần cuối hiển thị tổng tiền tạm tính.

Bước 3 — Thanh toán và tạo đơn hàng: Khách hàng nhấn nút "Đặt hàng ngay" ở cuối giỏ hàng. Hệ thống gửi yêu cầu POST /api/orders qua API Gateway → Order Service, lưu đơn hàng vào cơ sở dữ liệu và phát sự kiện ORDER_CREATED lên Kafka topic order-events.

- **Kết quả mong đợi:** Giỏ hàng được xóa sạch, hệ thống hiển thị thông báo Toast "Đặt hàng thành công! Nhà hàng đang xác nhận đơn.". Đơn hàng xuất hiện trong trang "Lịch sử đơn hàng" với trạng thái "Chờ xác nhận" cùng thanh tiến trình (Progress Bar) tương ứng.
- **Kết quả thực tế:** Đơn hàng được tạo thành công, Kafka phát sự kiện bất đồng bộ, tồn kho tự động được trừ phía Restaurant Service và thông báo tự động được tạo phía Notification Service.

Luồng quản lý nhà hàng và thực đơn (Owner Dashboard)



Hình 4 Giao diện quản lý nhà hàng

- **Kịch bản:** Chủ nhà hàng đăng nhập bằng tài khoản owner. Sau khi đăng nhập, thanh Header hiển thị nút "Dashboard". Nhấn vào để truy cập trang quản trị dành riêng cho Chủ nhà hàng tại đường dẫn /dashboard.
- **Thực thi:** Giao diện Owner Dashboard được tổ chức thành 3 tab chức năng:

Tab "Nhà hàng": Hiện thị form chỉnh sửa thông tin nhà hàng (Tên, Loại ẩm thực, Địa chỉ, Số điện thoại). Nếu chủ sở hữu quản lý nhiều nhà hàng, một bộ chọn dropdown hiển thị ở góc trên bên phải cho phép chuyển đổi nhanh giữa các nhà hàng mà không cần tải lại trang.

Tab "Thực đơn": Hiện thị bảng danh sách toàn bộ món ăn thuộc nhà hàng đang chọn, bao gồm thông tin: Tên món, Danh mục, Giá bán, Tồn kho và Trạng thái (Còn hàng/Hết hàng). Chủ nhà hàng có thể nhấn nút "Thêm món" để mở form thêm món mới, hoặc nhấn biểu tượng thùng rác để xóa món.

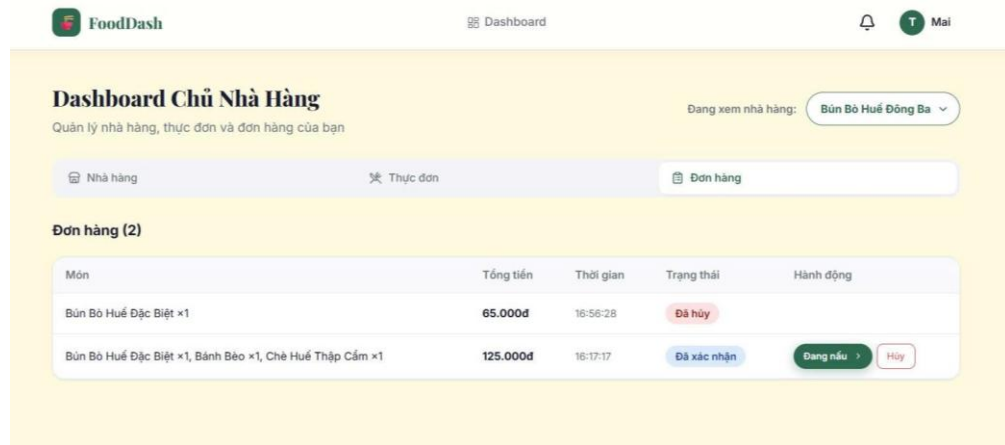
Tab "Đơn hàng": Hiện thị bảng danh sách tất cả các đơn hàng của nhà hàng. Mỗi đơn hàng hiển thị: Danh sách món đã đặt, Tổng tiền, Thời gian đặt, Trạng thái hiện tại. Chủ nhà hàng có thể nhấn nút để chuyển trạng thái đơn hàng theo quy trình: **Chờ duyệt** → **Đã xác nhận** → **Đang nấu** → **Đang giao** → **Hoàn thành**, hoặc nhấn nút "Hủy" để từ chối đơn hàng.

- **Kết quả mong đợi:** Dữ liệu nhà hàng, thực đơn và đơn hàng được hiển thị chính xác, liên kết đúng với tài khoản chủ sở hữu. Các thao tác CRUD (Tạo, Đọc, Sửa, Xóa) hoạt động ổn định.
- **Kết quả thực tế:** Chủ nhà hàng sở hữu nhiều nhà hàng (ví dụ: "Pizza Italia Express", "Phở Hà Nội 36") có thể chuyển đổi linh hoạt, dữ liệu thực đơn và đơn hàng được tải đúng theo từng nhà hàng.

Luồng tương tác thời gian thực qua hệ thống thông báo

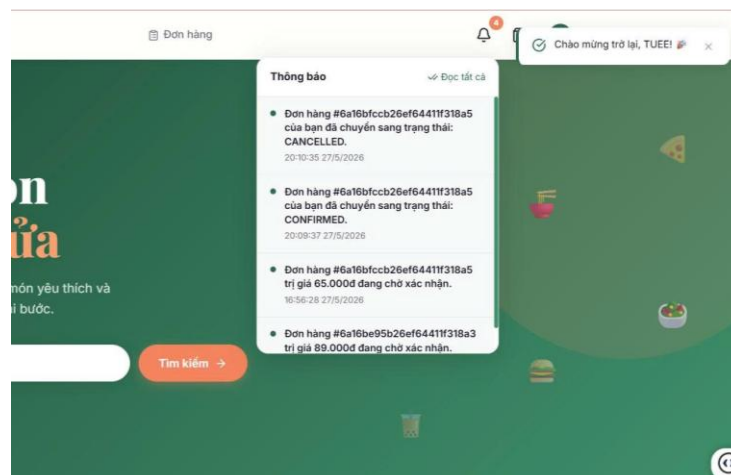
- **Kịch bản:** Mở hai cửa sổ trình duyệt song song — một bên đăng nhập với tài khoản **Khách hàng** và bên còn lại đăng nhập với tài khoản **Chủ nhà hàng**.
- **Thực thi:**
 1. Phía Khách hàng: Chọn một nhà hàng, thêm món ăn vào giỏ, nhấn "Đặt hàng ngay". Ngay sau khi đặt hàng thành công, biểu tượng chuông thông báo trên thanh Header hiển thị badge đếm số thông báo chưa đọc. Nhấn vào biểu tượng chuông, dropdown xuất hiện với nội dung: *"Đơn hàng mới được tạo thành công— Đơn hàng #... trị giá ...đ đang chờ xác nhận."*

2. Phía Chủ nhà hàng: Truy cập Dashboard → Tab "Đơn hàng", đơn hàng mới xuất hiện trong danh sách với trạng thái "Chờ duyệt". Nhấn nút "Đã xác nhận" để chuyển trạng thái.



Hình 5 Giao diện cập nhật trạng thái đơn hàng

3. Quay lại phía Khách hàng: Tải lại trang hoặc mở dropdown thông báo — xuất hiện thông báo mới: "Đơn hàng #... của bạn đã chuyển sang trạng thái: CONFIRMED."

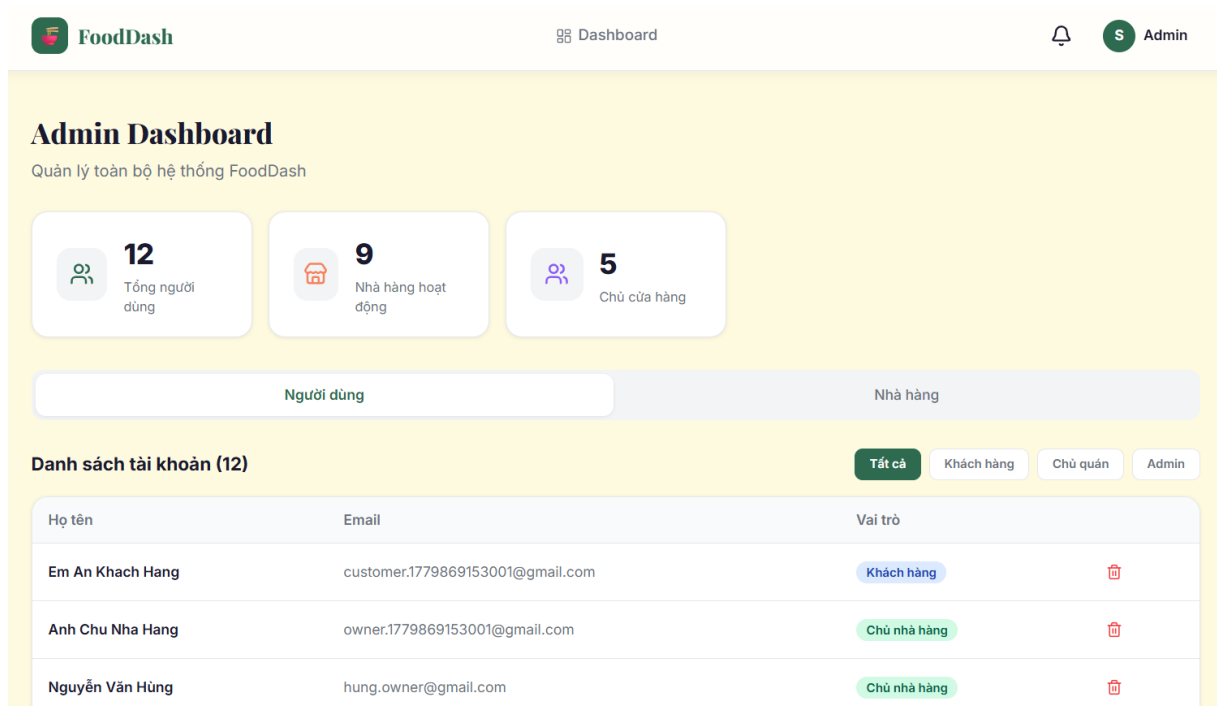


Hình 6 Giao diện thông báo

- **Kết quả mong đợi:** Thông báo được tạo tự động bởi Notification Service sau khi tiêu thụ sự kiện Kafka. Khách hàng nhận thông báo xác nhận mà không cần tải lại toàn bộ trang.
- **Kết quả thực tế:** Đạt yêu cầu. Sự kiện ORDER_CREATED và ORDER_STATUS_CHANGED được Kafka điều phối tới Notification Service (Consumer

Group: notification-group) và tạo bản ghi thông báo chính xác cho từng người dùng liên quan.

Luồng quản trị hệ thống (Admin Dashboard)

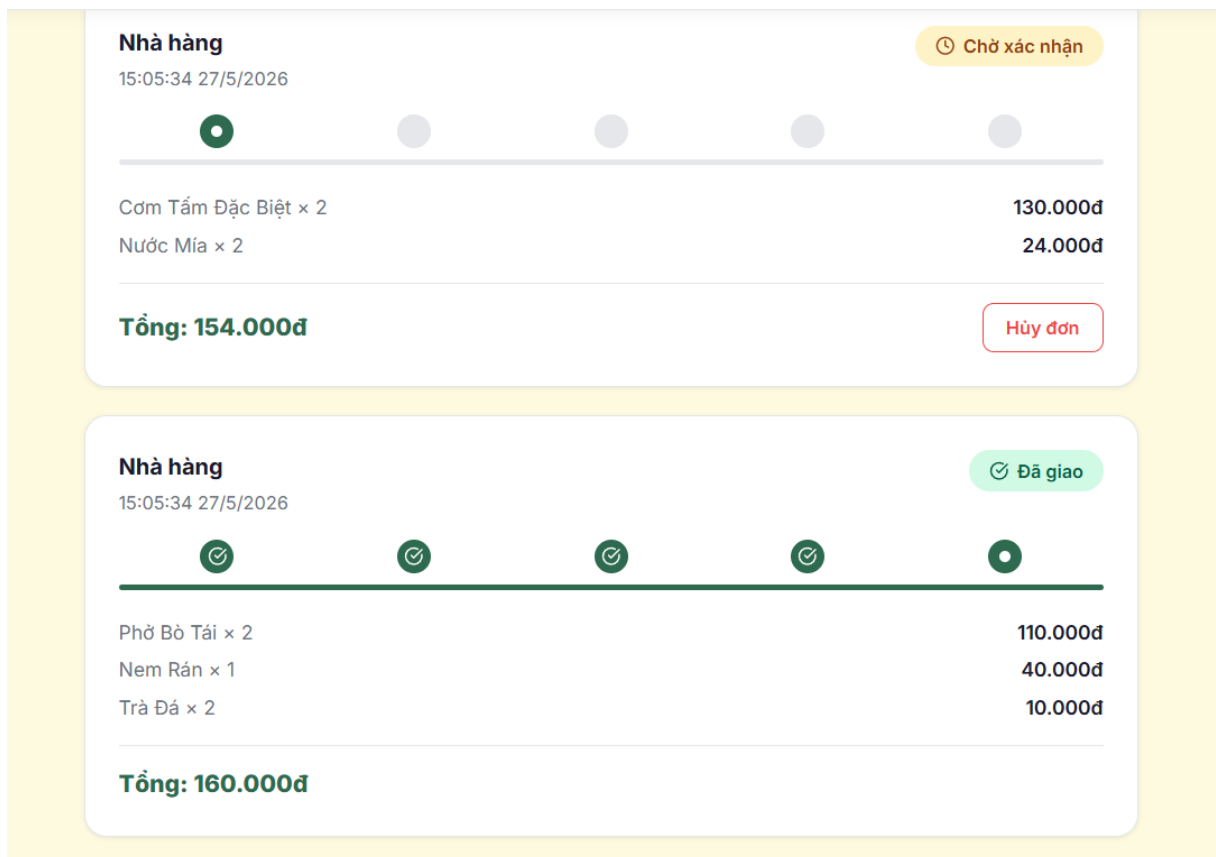


Hình 7 Giao diện Admin Dashboard

- **Kịch bản:** Quản trị viên đăng nhập bằng tài khoản Admin mặc định. Sau khi đăng nhập, thanh Header hiển thị nút "Dashboard". Truy cập trang quản trị tại đường dẫn /dashboard.
- **Thực thi:** Giao diện Admin Dashboard hiển thị:
 - **Bảng thống kê tổng quan:** 3 thẻ thống kê gồm Tổng người dùng, Nhà hàng đang hoạt động, Số lượng Chủ cửa hàng.
 - **Tab "Người dùng":** Bảng danh sách toàn bộ tài khoản đã đăng ký trên hệ thống, có thể lọc theo vai trò (Tất cả / Khách hàng / Chủ quán / Admin) bằng các nút bộ lọc. Mỗi dòng hiển thị Họ tên, Email, Badge vai trò (màu sắc phân biệt theo role) và nút Xóa.
 - **Tab "Nhà hàng":** Bảng danh sách toàn bộ nhà hàng, hiển thị Tên, Loại ẩm thực, Email chủ sở hữu, Trạng thái hoạt động (badge xanh/đỏ). Admin có quyền Bật/Tắt trạng thái hoạt động của bất kỳ nhà hàng nào.

- **Kết quả mong đợi:** Admin có toàn quyền xem, lọc, xóa tài khoản người dùng và bật/tắt trạng thái nhà hàng. Các thao tác được phản hồi bằng thông báo Toast ngay trên giao diện.
- **Kết quả thực tế:** Admin có thể xem danh sách đầy đủ, lọc theo vai trò, xóa tài khoản (trừ tài khoản Admin) và toggle trạng thái nhà hàng. Hệ thống phân quyền nghiêm ngặt: chỉ tài khoản có role admin mới truy cập được giao diện này.

Luồng xem lịch sử đơn hàng và hủy đơn (Customer)



Hình 8 Giao diện đơn hàng của khách

- **Kịch bản:** Sau khi đặt hàng, khách hàng nhấn nút "Đơn hàng" trên thanh Header để truy cập trang Lịch sử đơn hàng (/orders).
- **Thực thi:** Trang hiển thị danh sách tất cả đơn hàng đã đặt, mỗi đơn bao gồm:
 - Tên nhà hàng và thời gian đặt hàng.

- Badge trạng thái với màu sắc phân biệt (vàng — Chờ xác nhận, xanh dương — Đã xác nhận, tím — Đang chuẩn bị, xanh lá — Đang giao/Đã giao, đỏ — Đã hủy).
- **Thanh tiến trình (Progress Bar)** 5 bước trực quan: Chờ xác nhận → Đã xác nhận → Đang chuẩn bị → Đang giao → Đã giao.
- Danh sách món ăn kèm số lượng và đơn giá.
- Tổng tiền nổi bật.
- Nút "Hủy đơn" (chỉ hiển thị khi đơn hàng đang ở trạng thái "Chờ xác nhận").

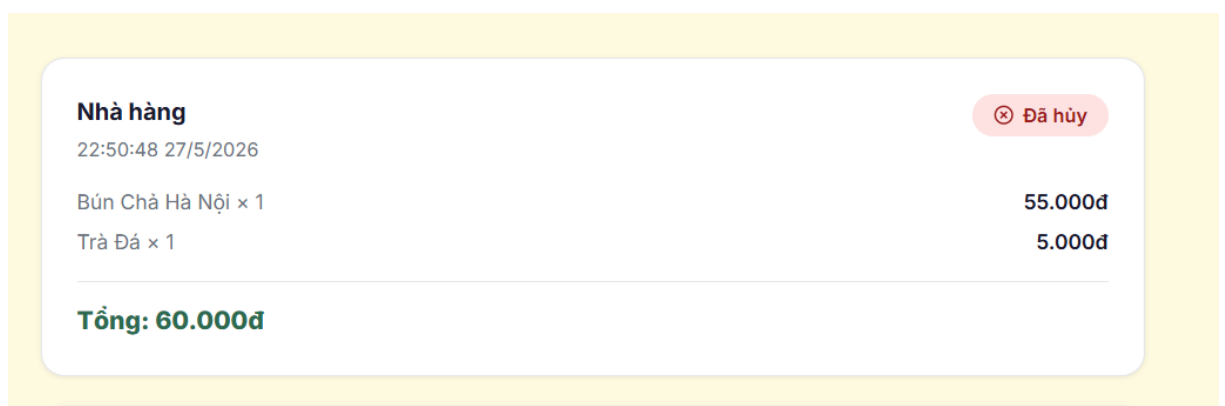
```

_id: ObjectId('6a16a5ce103a183992c6ad'  _id: ObjectId('6a16a5ce103a
restaurantId: ObjectId('6a16a5ce103a: restaurantId: ObjectId('6a1
name: "Bún Chả Hà Nội" name: "Trà Đá"
description: "Bún chả nướng than hoa description: "Trà đá truyền
price: 55000 price: 5000
category: "Bún" category: "Đồ uống"
isAvailable: true isAvailable: true
stock: 49 stock: 199
__v: 0 __v: 0
createdAt: 2026-05-27T08:05:34.605+0( createdAt: 2026-05-27T08:05
updatedAt: 2026-05-27T15:50:48.754+0( updatedAt: 2026-05-27T15:50

```

Hình 9 CSDL sau khi đặt món

Khách hàng nhấn nút "Hủy đơn" trên một đơn hàng đang chờ xác nhận. Hệ thống gửi yêu cầu hủy tới Order Service, Order Service phát sự kiện ORDER_CANCELLED lên Kafka.



Hình 10 Đơn hàng bị hủy

```

_id: ObjectId('6a16a5ce103a18392c6ad
restaurantId: ObjectId('6a16a5
name: "Bún Chả Hà Nội"
description: "Bún chả nướng th
price: 55000
category: "Bún"
isAvailable: true
stock: 50
__v: 0
createdAt: 2026-05-27T08:05:34.605+00
updatedAt: 2026-05-27T15:55:23

_id: ObjectId('6a16a5ce103a183992c6ad
restaurantId: ObjectId('6a16a5ce103a:
name: "Trà Đá"
description: "Trà đá truyền thống"
price: 5000
category: "Đồ uống"
isAvailable: true
stock: 200
__v: 0
createdAt: 2026-05-27T08:05:34.605+00
updatedAt: 2026-05-27T15:55:23.561+00

```

Hình 11 CSDL sau khi hủy đơn hàng

- **Kết quả mong đợi:** Đơn hàng chuyển sang trạng thái "Đã hủy" (badge đỏ), nút "Hủy đơn" biến mất. Phía backend, tồn kho món ăn tự động được hoàn trả bởi Restaurant Service (Kafka Consumer) và thông báo hủy đơn được tạo bởi Notification Service.
- **Kết quả thực tế:** Xác minh trong csdl cho thấy tồn kho món ăn trong collection menuitems đã được hoàn trả chính xác về giá trị ban đầu.

6.2.2 Kiểm thử tính bền bỉ và chịu lỗi của kiến trúc phân tán

Kịch bản gián đoạn cục bộ (Partial Outage):

- **Thực thi:** Tắt đột ngột container food-ordering-notification-service bằng lệnh docker stop food-ordering-notification-service trong khi khách hàng đang thao tác đặt hàng trên giao diện.
- **Kết quả:** Khách hàng vẫn đặt hàng thành công, giỏ hàng được xóa sạch, thông báo Toast "Đặt hàng thành công!" vẫn xuất hiện. Chủ nhà hàng vẫn nhìn thấy đơn hàng mới trên Dashboard. Tồn kho vẫn được trừ chính xác bởi Restaurant Service (Consumer Group riêng biệt). Sau khi khởi động lại container bằng docker start food-ordering-notification-service, Kafka tự động gửi lại các sự kiện chưa được xử lý (nhờ cơ chế offset tracking), và thông báo được tạo bù cho khách hàng.

- **Nhận xét:** Điều này chứng minh hệ thống không mắc lỗi **Single Point of Failure** — sự cố tại Notification Service không gây ảnh hưởng đến luồng đặt hàng và quản lý tồn kho cốt lõi.

Kịch bản giới hạn tần suất truy cập (Rate Limiting)

- **Thực thi:** API Gateway Nginx được cấu hình giới hạn 50 request/giây cho luồng xác thực (auth_limit) và 100 request/giây cho các luồng nghiệp vụ khác (general_limit). Thực hiện gửi liên tục hàng trăm request đồng thời tới endpoint đăng nhập.
- **Kết quả:** Sau khi vượt ngưỡng, Nginx tự động trả về mã lỗi **HTTP 429 Too Many Requests** với phản hồi JSON chuẩn hóa: {"success": false, "message": "Too many requests. Please slow down and try again shortly."}. Giao diện Frontend hiển thị thông báo lỗi tương ứng.
- **Nhận xét:** Cơ chế này bảo vệ hệ thống khỏi các cuộc tấn công Brute-Force và DDoS cơ bản ngay tại tầng Gateway, trước khi yêu cầu được chuyển tiếp tới microservice phía sau.

6.3 Đánh giá ưu điểm của hệ thống

Quá trình triển khai và kiểm thử thực tiễn trên giao diện người dùng đã khẳng định tính đúng đắn khi lựa chọn kiến trúc Microservices kết hợp Event-Driven Architecture cho hệ thống FoodDash:

- **Tách biệt mối quan tâm:** Mỗi microservice chỉ giải quyết một bài toán nghiệp vụ duy nhất với cơ sở dữ liệu riêng biệt. Mã nguồn của từng dịch vụ gọn nhẹ, dễ bảo trì. Sự cố tại Notification Service được cô lập hoàn toàn, không gây ảnh hưởng đến luồng đặt hàng cốt lõi của Order Service.
- **Khả năng mở rộng linh hoạt:** Trong các kịch bản lượng đơn hàng tăng đột biến, quản trị viên chỉ cần nhân bản riêng lẻ container Order Service để đáp ứng tải, tiết kiệm tối đa tài nguyên so với việc nhân bản toàn bộ ứng dụng nguyên khối (Monolithic).

- **Tối ưu hiệu năng với luồng sự kiện bất đồng bộ:** Các tác vụ phụ trợ (trừ/hoàn kho, gửi thông báo) được đẩy vào Kafka xử lý ngầm. Nhờ đó, API POST /api/orders phản hồi ngay lập tức — khách hàng nhận kết quả đặt hàng mà không cần chờ các tác vụ phía sau hoàn thành.
- **Bảo mật nhiều tầng:** API Gateway Nginx là tầng bảo vệ đầu tiên với Rate Limiting và xác thực JWT động (auth_request). Tại tầng microservice, middleware authorize() kiểm tra phân quyền theo role (RBAC). Tại tầng Frontend, component ProtectedRoute ngăn chặn truy cập giao diện không phù hợp với vai trò.
- **Tự động hóa triển khai hoàn toàn:** Docker Compose cho phép khởi chạy toàn bộ container chỉ bằng một lệnh duy nhất (docker-compose up -d --build), đảm bảo tính tái lập và nhất quán trên mọi máy tính phát triển.

6.4 Hạn chế và hướng phát triển trong tương lai

Mặc dù hệ thống đã hoàn thành đầy đủ các chức năng cốt lõi và vượt qua các kịch bản kiểm thử tích hợp, vẫn còn một số hạn chế cần tiếp tục cải thiện để đáp ứng yêu cầu triển khai thực tế ở quy mô lớn.

Quản lý giao dịch phân tán

Hiện tại hệ thống mới sử dụng cơ chế xử lý sự kiện cơ bản nên vẫn tồn tại khả năng dữ liệu không nhất quán tạm thời khi Consumer gặp lỗi. Trong tương lai có thể áp dụng Saga Pattern kết hợp Outbox Pattern để nâng cao độ tin cậy cho các giao dịch phân tán.

Tích hợp thanh toán trực tuyến

Hệ thống hiện chỉ hỗ trợ thanh toán khi nhận hàng (COD). Các phiên bản tiếp theo có thể tích hợp cổng thanh toán điện tử như VNPay hoặc MoMo để nâng cao trải nghiệm người dùng.

Hoàn thiện hệ thống giám sát

Hiện tại việc ghi log chủ yếu diễn ra cục bộ trong container. Trong tương lai có thể triển khai ELK Stack hoặc Prometheus kết hợp Grafana nhằm phục vụ giám sát và truy vết phân tán.

Triển khai trên nền tảng Cloud

Hệ thống mới được triển khai cục bộ bằng Docker Compose. Trong môi trường Production, có thể triển khai lên Kubernetes trên AWS hoặc Google Cloud nhằm nâng cao khả năng mở rộng và tính sẵn sàng cao.

Tự động hóa CI/CD

Quy trình build và deploy vẫn còn thực hiện thủ công. Trong tương lai cần tích hợp GitHub Actions hoặc GitLab CI/CD để tự động hóa toàn bộ quá trình kiểm thử và triển khai hệ thống.

TÀI LIỆU THAM KHẢO

- [1] Newman, S. (2021). *Building Microservices: Designing Fine-Grained Systems* (2nd ed.). O'Reilly Media. (Tài liệu kinh điển về cách thiết kế, phân tách và quản trị hệ thống Microservices).
- [2] Richardson, C. (2018). *Microservices Patterns: With examples in Java*. Manning Publications. (Tài liệu cốt lõi về các pattern trong Microservices như API Gateway, Database-per-service, và Event-Driven).
- [3] Kleppmann, M. (2017). *Designing Data-Intensive Applications: The Big Ideas Behind Reliable, Scalable, and Maintainable Systems*. O'Reilly Media. (Tài liệu tham khảo cho phần cơ lập dữ liệu và thiết kế hệ thống phân tán).
- [4] Narkhede, N., Shapira, G., & Palino, T. (2017). *Kafka: The Definitive Guide: Real-Time Data and Stream Processing at Scale*. O'Reilly Media. (Tài liệu nền tảng về cách hoạt động của Apache Kafka, Topic và Consumer Group).

PHỤ LỤC

Mã nguồn dự án: <https://github.com/Cinder871169/HTPT>

